# 📋 Aide-Mémoire - Présentation BookTracker

## 🚀 DÉMARRAGE RAPIDE (3 commandes)

Option 1: Automatique (RECOMMANDÉ)

```
.\start.ps1
```

☑ Tout démarre automatiquement !

Option 2: Manuel

```
# 1. Backend
docker-compose up -d

# 2. Frontend (dans un autre terminal)
cd frontend
npm run dev
```

🌐 URL à ouvrir:

**http://localhost:3000**

---

## 🛑 ARRÊT RAPIDE

```
.\stop.ps1
```

---

## 🎯 SCÉNARIO DE DÉMO (5 minutes)

1 Inscription/Connexion (30s)

- Créer compte: `jury@booktracker.com` / `DemoJury2026!`
- **Dire**: "JWT authentication avec Spring Security"

2 Parcourir Catalogue (1min)

- Montrer liste de livres
- Utiliser recherche/filtres
- **Dire**: "Book Service avec pagination et recherche full-text"

3 Ajouter à ma Bibliothèque (1min)

- Choisir un livre
- Cliquer "Ajouter à ma bibliothèque"
- Marquer "En cours de lecture"
- **Dire**: "Tracker Service gère l'état personnel de lecture"

### 4️⃣ Mettre à jour Progression (1min)

- Entrer nombre de pages lues (ex: 50/300)
- Voir barre de progression
- **Dire**: "Suivi temps réel de la progression"

### 5️⃣ Recommandations (1min)

- Aller dans "Recommandations"
- Montrer suggestions personnalisées
- **Dire**: "Recommendation Service avec Redis cache"

### 6️⃣ Ajouter un Avis (30s)

- Noter un livre (étoiles)
- Écrire commentaire
- **Dire**: "Système d'avis communautaire"

---

## 🏗️ ARCHITECTURE (à mentionner)

```
Frontend (React) → API Gateway → 4 Microservices
                                  ↓
                          4 PostgreSQL + Redis
```

Services:

- **Port 3000**: Frontend React
- **Port 8080**: API Gateway (point d'entrée)
- **Port 8081**: Auth Service (authentification JWT)
- **Port 8082**: Book Service (catalogue)
- **Port 8083**: Tracker Service (suivi lecture)
- **Port 8084**: Reco Service (recommandations)

---

## 💡 POINTS TECHNIQUES À MENTIONNER

Technologies:

- ☑ **Java 21 LTS** (migration récente, 100% tests passés)
- ☑ **Spring Boot 3.2.3** (framework moderne)
- ☑ **React 18 + TypeScript** (frontend typé)
- ☑ **PostgreSQL 16** (base de données robuste)

- ☑ **Redis** (cache haute performance)
- ☑ **Docker** (conteneurisation)

Architecture:

- ☑ **Microservices** (scalabilité + maintenance)
- ☑ **API Gateway Pattern** (routing centralisé)
- ☑ **Database per Service** (isolation données)
- ☑ **JWT Stateless Auth** (pas de sessions serveur)

---

## ⚠ DÉPANNAGE EXPRESS

Si erreur connexion:

```
# Vérifier services backend
docker ps

# Voir logs API Gateway
docker logs booktracker-gateway --tail 20
```

Si frontend ne démarre pas:

```
cd frontend
npm install
npm run dev
```

Si port déjà utilisé:

```
# Arrêter tout
docker-compose down
# Redémarrer
docker-compose up -d
```

---

## 📊 CHIFFRES À CONNAÎTRE

- **10 conteneurs** Docker (5 services + 4 DB + 1 Redis)
- **5 microservices** Spring Boot indépendants
- **4 bases** PostgreSQL dédiées
- **100% tests** passés après migration Java 21
- **Architecture REST** avec API Gateway
- **JWT tokens** pour sécurisation

---

## 🎤 PHRASES CLÉS POUR LE JURY

### Introduction:

> "BookTracker est une application de suivi de lecture utilisant une architecture microservices moderne. Elle permet aux lecteurs de gérer leur bibliothèque, suivre leur progression, et recevoir des recommandations personnalisées."

### Architecture:

> "Nous avons choisi une architecture microservices pour la scalabilité et la maintenabilité. Chaque service est indépendant avec sa propre base de données, communiquant via une API Gateway centralisée."

### Technologies:

> "Le backend utilise Java 21 LTS et Spring Boot 3.2.3, garantissant performance et sécurité. Le frontend React offre une expérience utilisateur moderne et réactive."

### Sécurité:

> "L'authentification est gérée via JWT tokens avec Spring Security. Les mots de passe sont hashés avec BCrypt, et toutes les routes sensibles nécessitent une authentification."

### Migration Java 21:

> "Nous venons de migrer de Java 17 vers Java 21 LTS, bénéficiant des dernières optimisations de performance. Tous les tests passent à 100%."

---

## ☑ CHECKLIST PRÉ-PRÉSENTATION

**30 min avant:**

- ☐ Démarrer Docker Desktop
- ☐ Tester `.\start.ps1`
- ☐ Vérifier http://localhost:3000 fonctionne
- ☐ Fermer applications inutiles
- ☐ Désactiver notifications
- ☐ Brancher alimentation

**5 min avant:**

- ☐ Ouvrir PowerShell dans le projet
- ☐ Avoir navigateur prêt (onglet vide)
- ☐ Avoir ce fichier ouvert pour référence
- ☐ Prendre une grande respiration 😊

---

## 🎯 TIMING IDÉAL

- **Démarrage**: 2-3 minutes
- **Démonstration**: 5-6 minutes
- **Questions**: 3-4 minutes
- **Total**: ~12 minutes

---

**Bonne chance !** 🚀